Distributed Systems and Cloud Computing

INTE 22253

MODULE OVERVIEW



Distributed System Models



IPC & Synchronization



Time & Consistency



Replication & Fault Tolerance



Distributed FS & Security



Cloud Computing



Virtualization & Management



Cloud Storage Technologies



Distributed Transactions



Multi-Agent Systems



Distributed Search Engines



P2P & Blockchain

50%

CONTINUOUS ASSESSMENT

- Assignment
- Mini-Project

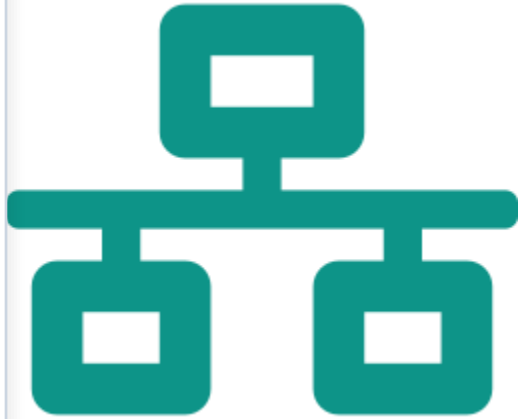
50%

FINAL EXAM

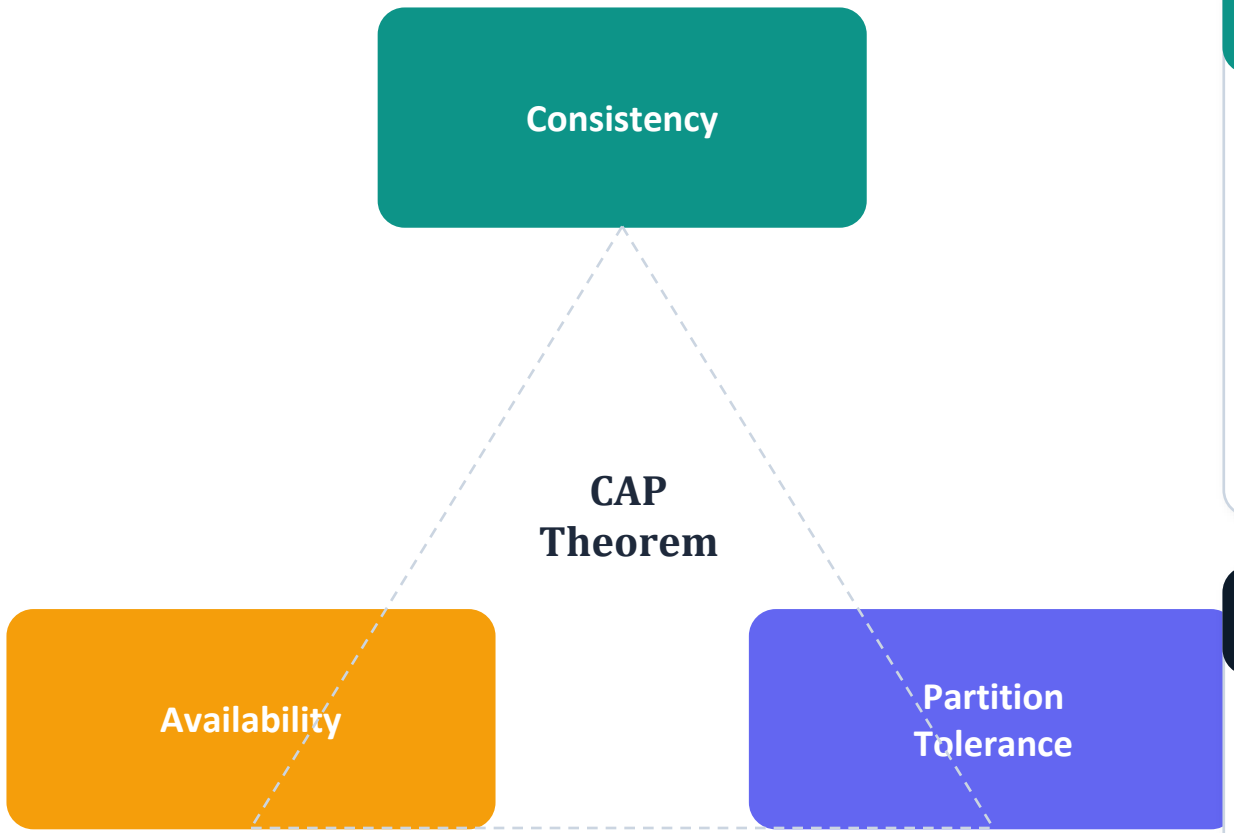
- End-of-Course
- Time: 2 Hours
- Closed Book

Fundamental Models for Distributed Systems

1. What is a distributed system? Goals & challenges
2. Architectural models: client-server, peer-to-peer, microservices
3. Interaction models: synchronous vs. asynchronous communication
4. Failure models: crash, omission, timing, Byzantine faults
5. Security models: threats and trust hierarchies
6. Key properties: transparency, openness, scalability, reliability



Consistency, Replication & Fault Tolerance



Consistency Models

- Strong / Sequential Consistency
- Eventual Consistency
- Causal Consistency
- Read-your-Writes, FIFO

Replication Strategies

- Active vs. Passive Replication
- Primary-Backup Protocol
- Quorum-based approaches

Distributed File Systems & Security



DFS Fundamentals

- Naming & Transparency
- Caching strategies
- NFS, AFS, HDFS overview
- File replication & availability



Access Control

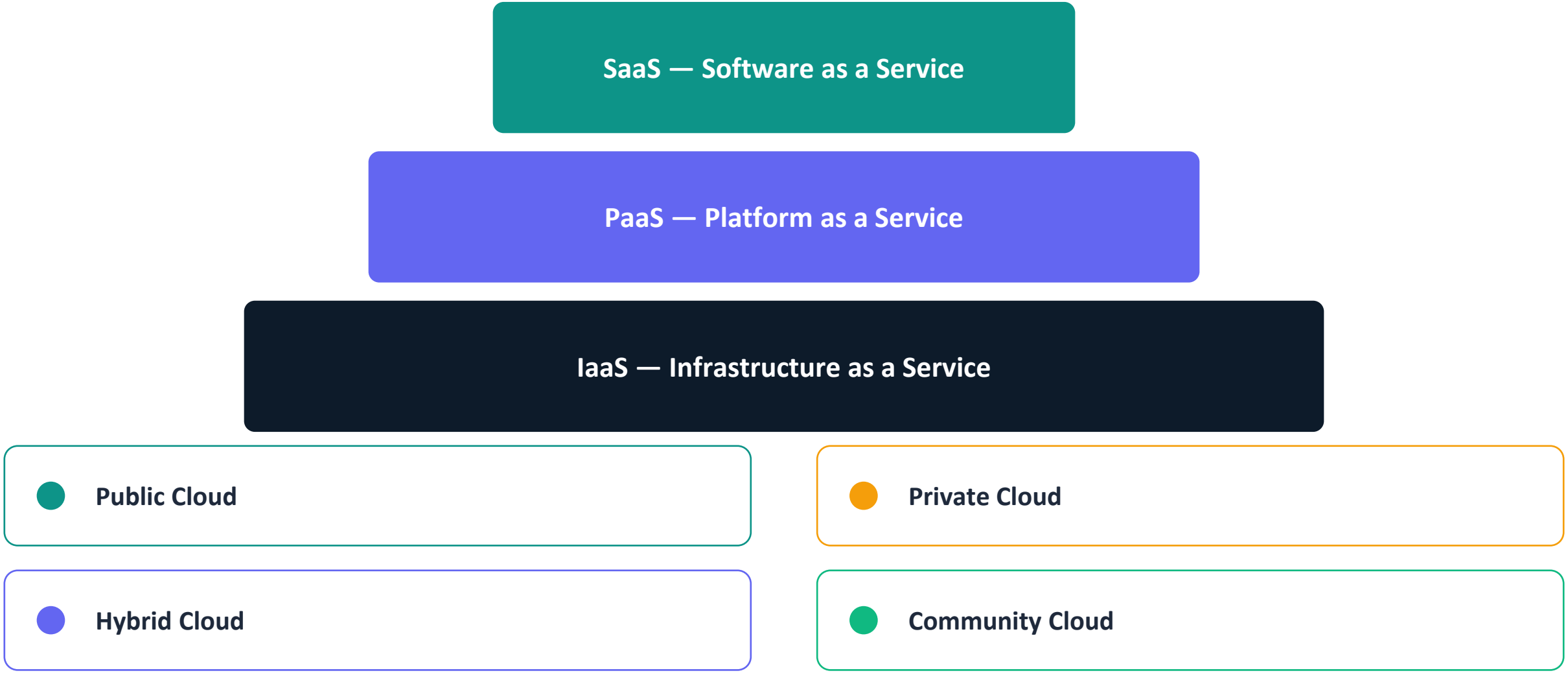
- Authentication & Authorization
- ACLs & Capability Lists
- Kerberos authentication
- OAuth & token-based auth



Security Threats

- Eavesdropping & MITM
- Replay attacks
- Denial of Service (DoS)
- Secure channels (TLS/SSL)

Cloud Computing Fundamentals



Virtualization, Cloud Infrastructure & Management

Virtualization Types

- Full Virtualization (VMware, KVM)
- Para-Virtualization (Xen)
- OS-level Containers (Docker, LXC)
- Hardware-assisted Virtualization

Cloud Infrastructure

- Hypervisors (Type 1 & 2)
- Software-Defined Networking
- Load Balancing & Auto-scaling
- Resource Pooling

Deployment & Management

- CI/CD Pipelines
- Infrastructure as Code (Terraform)
- Kubernetes Orchestration
- Monitoring & Observability

Cloud Storage Technologies & Distributed File Systems

Object Storage

Amazon S3, Azure Blob
Immutable objects, REST API

Block Storage

AWS EBS, SAN
Low-latency, database workloads

File Storage

NFS, SMB, EFS
Shared file system access

HDFS

Hadoop Distributed FS
Big data batch processing

GFS / Colossus

Google File System
Large sequential reads/writes

Ceph / GlusterFS

Open-source distributed FS
Scale-out storage

Distributed Transactions & Multi-Agent Systems

Distributed Transactions

- ACID properties in distributed settings
- Two-Phase Commit (2PC) Protocol
- Three-Phase Commit (3PC)
- Distributed Deadlock detection
- Optimistic vs Pessimistic Concurrency
- BASE: Basically Available, Soft-state, Eventually consistent

Multi-Agent Systems

- Agent definition: autonomous, reactive
- Types: reactive, deliberative, hybrid
- Agent Communication Languages (ACL)
- Coordination & negotiation protocols
- FIPA standards & architectures
- Applications: autonomous vehicles, IoT

PART 11 & 12

Distributed Search, P2P Systems & Blockchain



Distributed Search Engines

- Web crawling & indexing at scale
- Inverted index structures
- MapReduce for search
- Ranking: PageRank algorithm
- Challenges: freshness, coverage



Peer-to-Peer Systems

- Unstructured P2P (Gnutella)
- Structured P2P — DHT (Chord, Kademlia)
- Content distribution (BitTorrent)
- Reputation & trust management
- Legal & ethical considerations



Blockchain Technologies

- Distributed ledger concepts
- Consensus: PoW, PoS, PBFT
- Smart contracts (Ethereum)
- Permissioned blockchains (Hyperledger)
- Applications beyond cryptocurrency

LEARNING OUTCOMES

LO1 Describe and compare fundamental distributed system architectures and communication models

LO2 Apply synchronization algorithms and reason about time ordering in distributed environments

LO3 Evaluate consistency models and design fault-tolerant replicated systems

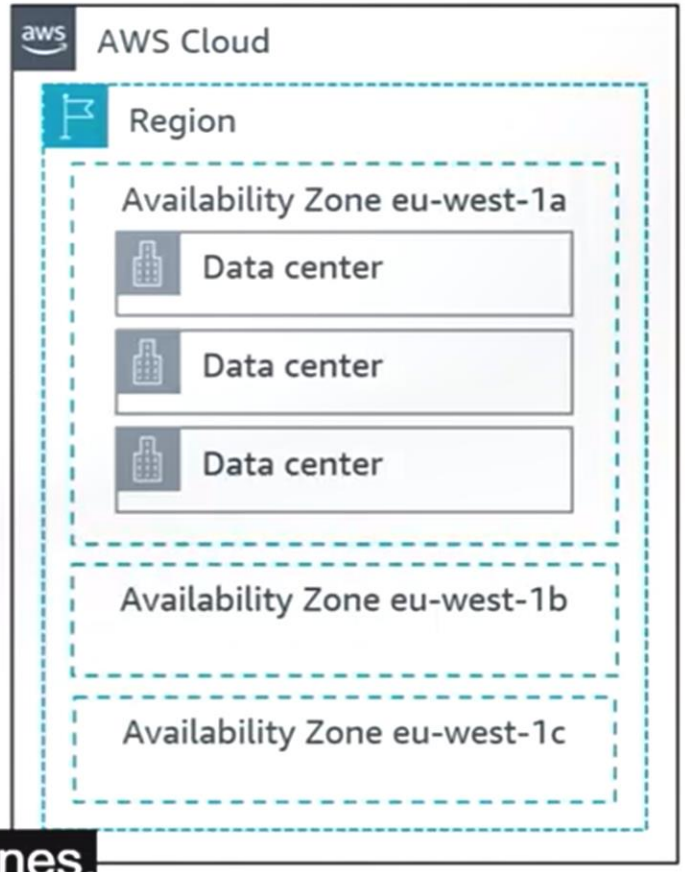
LO4 Design and configure cloud infrastructure using IaaS, PaaS, and SaaS models

LO5 Implement and manage distributed file systems and cloud storage solutions

LO6 Analyse distributed transactions, multi-agent coordination, P2P, and blockchain technologies

Selecting Availability Zones

- Each Availability Zone includes the following:
  - It is made up of one or more data centers.
  - It is designed for fault isolation.
  - It is interconnected with other Availability Zones in a Region using high-speed private links.
- For certain services, you can choose your Availability Zones.
- AWS recommends replicating across Availability Zones for resiliency.



to two Availability Zones.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Introduction to Distributed Systems



Definition • Characteristics • Advantages • Challenges • Real-World Examples

OVERVIEW

What We'll Cover



1

Definition & Characteristics

What makes a system "distributed" and the core properties every one of them shares.



2

Advantages & Challenges

The compelling benefits — and the hard engineering trade-offs that come with them.



3

Real-World Examples

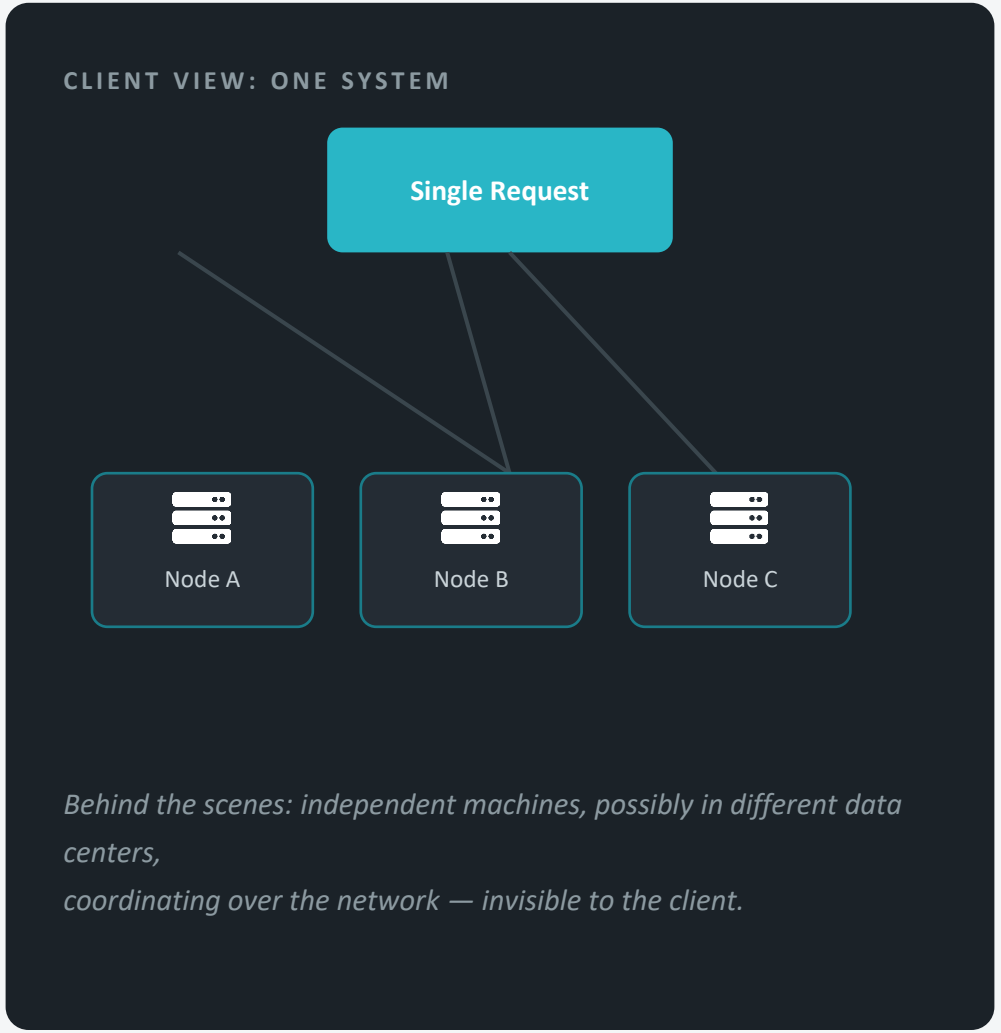
How Google, AWS, and global banking systems put these ideas into production.

SECTION 1 · DEFINITION

What Is a Distributed System?

A collection of independent computers that appears to its users as a single coherent system — working together, communicating over a network, to achieve a common goal.

- Multiple autonomous computers ("nodes") that do not share physical memory
- Nodes coordinate solely by passing messages across a network
- The system presents a single, unified view to the end user or application
- No global clock and no shared state — coordination must be explicit



SECTION 1 · CHARACTERISTICS

Core Characteristics

Resource Sharing

Hardware, data, and services are shared across nodes rather than duplicated on one machine.

Scalability

Capacity grows by adding more nodes (horizontal scaling) rather than upgrading one machine.

Concurrency

Many nodes process requests and operations simultaneously, independent of one another.

Fault Tolerance

The system keeps running correctly even when individual nodes fail or disconnect.

No Global Clock

Nodes can't rely on perfectly synchronized time, complicating event ordering.

Transparency

Distribution is hidden — users interact with one logical system, not many machines.

SECTION 2 · ADVANTAGES

Why Go Distributed?

Scalability

Add more machines on demand to handle growth — no ceiling imposed by a single server's hardware.

Fault Tolerance

If one node fails, others take over. There's no single point of failure for the whole system.

Performance

Workloads run in parallel across many nodes, cutting response time for large or complex tasks.

Resource Efficiency

Hardware, storage, and bandwidth are pooled and shared rather than sitting idle on one machine.

Geographic Reach

Nodes placed near users worldwide reduce latency and keep services available regionally.

Cost-Effective Growth

Scaling out with commodity servers is often cheaper than scaling up one powerful machine.

SECTION 2 · CHALLENGES

The Hard Trade-offs

Complexity

More moving parts means harder design, debugging, deployment, and testing.

Partial Failures

Some nodes can fail while others keep running — much trickier than an all-or-nothing crash.

Consistency

Keeping data in sync across nodes is hard, especially under concurrent updates.

Synchronization

No global clock makes ordering events and coordinating actions genuinely difficult.

Security

More network communication paths mean a larger attack surface to defend.

Network Reliability

Latency, partitions, and message loss are normal — and must be designed for, not assumed away.

SECTION 2 · A KEY TRADE-OFF

The CAP Theorem

In a distributed system, you can only fully guarantee two of these three properties at once.



Consistency

Every node sees the same data at the same time.



Availability

Every request gets a response, even if some nodes are down.



Partition Tolerance

The system keeps working despite network failures between nodes.

In practice, network partitions will happen — so real systems choose between consistency and availability when one occurs. Banking systems often favor consistency; social feeds often favor availability.

SECTION 3 · REAL-WORLD EXAMPLE

Google: Search at Planetary Scale

Google processes hundreds of billions of search queries — each one touching a global network of data centers, not one giant computer.

Massive Server Farms

Millions of commodity servers across data centers worldwide, working as one logical system.

Distributed File System (GFS)

Files are split into chunks and replicated across many machines for durability and speed.

MapReduce

Huge computations are split into small tasks, run in parallel, then combined — built for failure-prone clusters.

Why it matters: a query answered in milliseconds is actually the product of thousands of machines cooperating behind the scenes.

SECTION 3 · REAL-WORLD EXAMPLE

AWS: Cloud Infrastructure as a Service

AWS rents out distributed computing itself — letting any business run global, fault-tolerant infrastructure without owning a single server.

Global Regions & Zones

Data centers grouped into regions and availability zones, so apps can survive losing an entire facility.

Elastic Compute (EC2)

Servers spin up or down automatically as demand changes, distributing load across many instances.

Distributed Storage (S3)

Objects are replicated across multiple devices and facilities for extreme durability.

Why it matters: "the cloud" is really a vast distributed system — AWS just makes its complexity invisible to the customer.

SECTION 3 · REAL-WORLD EXAMPLE

Banking Systems: Consistency at Stake



Banks run distributed systems across branches, ATMs, and card networks — where correctness matters more than almost anywhere else.



Strong Consistency

A withdrawal at an ATM must reflect instantly everywhere — no two systems can show different balances.



Distributed Transactions

Transfers between accounts at different banks use protocols ensuring both sides commit, or neither does.



High Availability

Core banking and card networks must stay up nationally, even if individual data centers fail.

Why it matters: banking shows the CAP trade-off in action — consistency is non-negotiable, even at some cost to availability.

KEY TAKEAWAYS

Distributed Systems, in Summary

Many machines, one experience

Independent nodes cooperate over a network to appear as a single, unified system.

Power comes with complexity

Scalability and fault tolerance are real wins — but they demand careful design for failure, consistency, and timing.

Trade-offs are the design

CAP theorem reminds us: consistency, availability, and partition tolerance can't all be maximized at once.

It's already everywhere

Google, AWS, and global banking show distributed systems aren't theoretical — they run daily life.

Thank you


Attendance

Q & A

Thank You